Grupo 6: Oldrin BONILLA, Gandhi MORA, Tais RODRIGUEZ

El desarrollo del presente trabajo se organizó de manera colaborativa, asignando responsabilidades específicas a cada integrante del grupo para optimizar la eficiencia y calidad del resultado final. Stan fue responsable de la sección de Uso de Algoritmos de Búsqueda, desarrollando la representación del laberinto como grafo, la implementación de los algoritmos (BFS, DFS y A*) y el análisis comparativo de sus resultados. Santiago se encargó de la implementación y análisis de la técnica de Optimización por Colonias de Hormigas (ACO), incluyendo la ejecución, corrección y ajuste de parámetros del modelo. Por su parte, Tais estuvo a cargo de la redacción del ensayo, la integración y mejora de la calidad del contenido escrito, así como de la elaboración final del documento en formato PDF.

I. USO DE ALGORITMOS DE BÚSQUEDA

A. Leer el laberinto y representarlo como un grafo

El problema del laberinto se resolvió mediante su transformación a un grafo, lo que permite aplicar algoritmos clásicos de búsqueda. En esta representación, cada celda transitable del laberinto corresponde a un nodo, mientras que las conexiones entre celdas adyacentes (arriba, abajo, izquierda y derecha) representan las aristas. Las celdas que contienen paredes (#) no forman parte del grafo, ya que no son transitables.

El punto de inicio (E) se define como el nodo inicial, y el punto de salida (S) como el nodo objetivo. A partir de esta estructura, el problema se convierte en encontrar un camino entre dos nodos dentro de un grafo, lo que permite aplicar distintos algoritmos de búsqueda.
Para construir el grafo, se recorre el laberinto celda por celda, identificando las posiciones válidas y estableciendo sus vecinos accesibles. De esta manera, cada nodo contiene información sobre las posibles transiciones a otras celdas, lo que define el espacio de búsqueda.
Esta representación es fundamental, ya que permite abstraer el problema espacial del laberinto en una estructura matemática que facilita la aplicación de algoritmos como BFS, DFS y A*, permitiendo analizar su comportamiento y eficiencia en la búsqueda de soluciones.

Algoritmo	Tipo de búsqueda	Garantiza solución óptima	Uso de heurística	Eficiencia en exploración	Observación
BFS	No informada (por niveles)	Sí	No	Baja–Media	Explora exhaustivamente el grafo, asegura el camino más corto
A*	Informada	Sí	Sí (distancia Manhattan)	Alta	Dirige la búsqueda hacia el objetivo, reduce nodos explorados
DFS	No informada (profundidad)	No	No	Variable	Puede ser rápido, pero no garantiza el camino más corto

Grupo 6: Oldrin BONILLA, Gandhi MORA, Tais RODRIGUEZ

Comparación de algoritmos — Laberinto 1 (pequeño)

BFS | Pasos: 14 | Nodos explorados: 43 | Tiempo: 0.048 ms

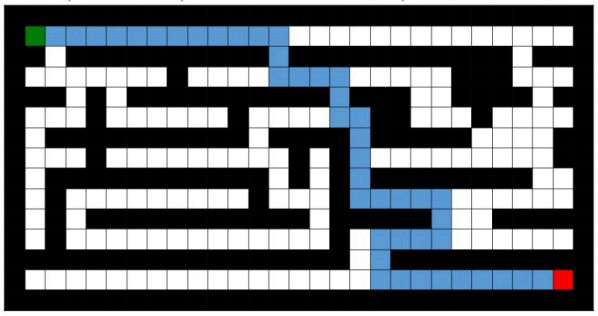
A* | Pasos: 14 | Nodos explorados: 36 | Tiempo: 0.090 ms

DFS | Pasos: 30 | Nodos explorados: 34 | Tiempo: 0.027 ms

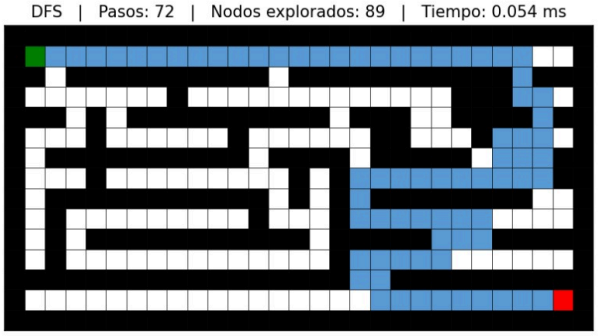
Comparación de algoritmos — Laberinto 2 (mediano)

BFS | Pasos: 44 | Nodos explorados: 183 | Tiempo: 0.088 ms

A* | Pasos: 44 | Nodos explorados: 140 | Tiempo: 0.197 ms



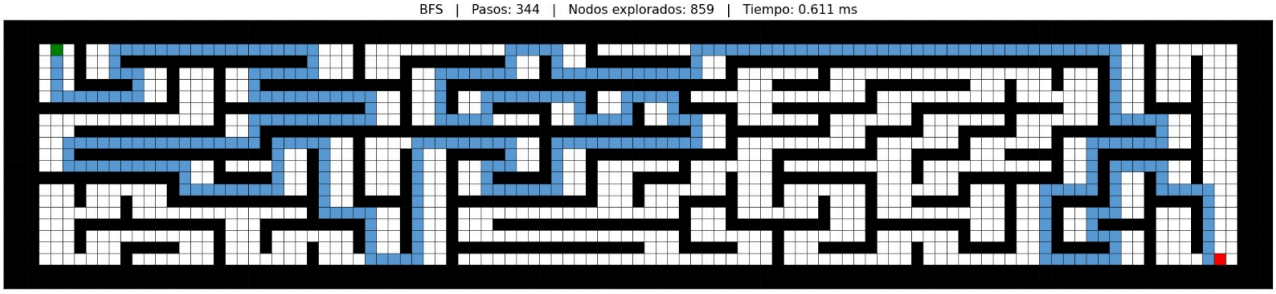
DFS | Pasos: 72 | Nodos explorados: 89 | Tiempo: 0.054 ms



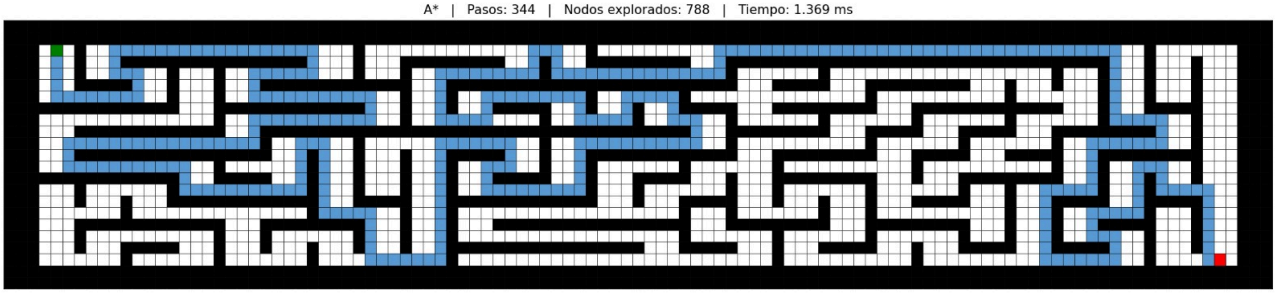
Grupo 6: Oldrin BONILLA, Gandhi MORA, Tais RODRIGUEZ

Comparación de algoritmos — Laberinto 3 (grande)

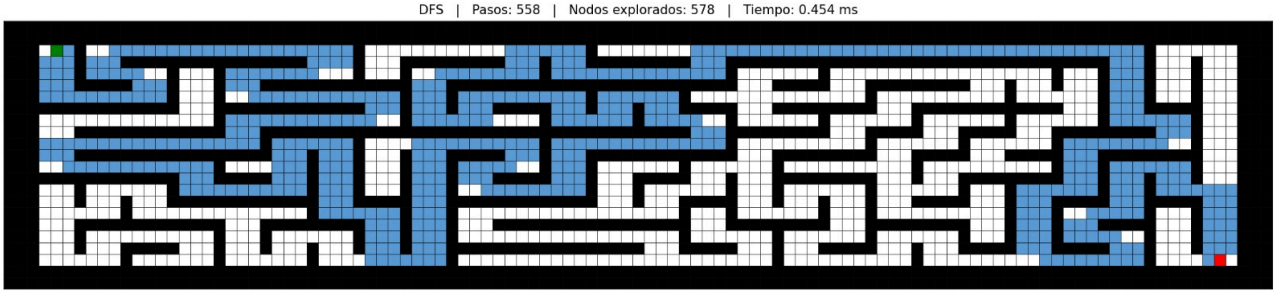
BFS | Pasos: 344 | Nodos explorados: 859 | Tiempo: 0.611 ms



A* | Pasos: 344 | Nodos explorados: 788 | Tiempo: 1.369 ms



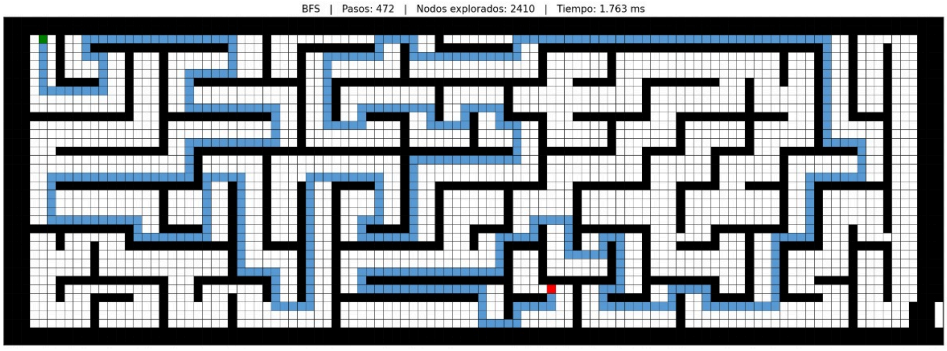
DFS | Pasos: 558 | Nodos explorados: 578 | Tiempo: 0.454 ms



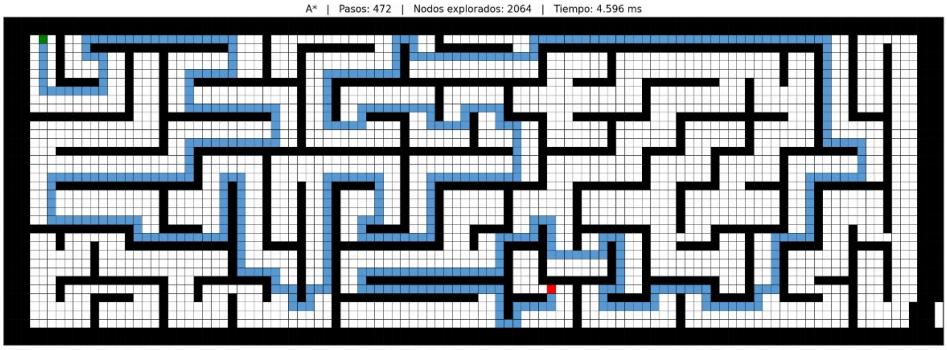
Grupo 6: Oldrin BONILLA, Gandhi MORA, Tais RODRIGUEZ

Comparación de algoritmos — Laberinto 4 (grande extendido)

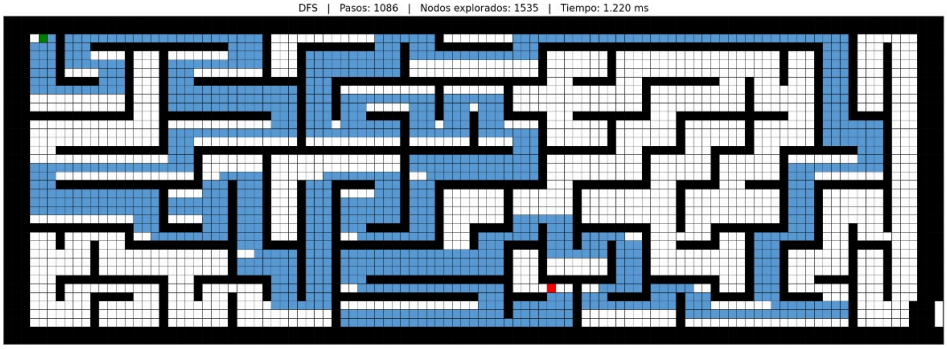
BFS | Pasos: 472 | Nodos explorados: 2410 | Tiempo: 1.763 ms



A* | Pasos: 472 | Nodos explorados: 2064 | Tiempo: 4.596 ms



DFS | Pasos: 1086 | Nodos explorados: 1535 | Tiempo: 1.220 ms



B. Aplicar algoritmos de búsqueda

Sí, es posible establecer métricas claras para evaluar el desempeño de los algoritmos de búsqueda en este problema. En particular, se consideran tres dimensiones principales: la optimalidad del camino (M1), la eficiencia en la exploración (M2) y el tiempo de ejecución (M3). La métrica más relevante es la optimalidad, ya que el objetivo fundamental del problema es encontrar un camino válido y lo más corto posible desde el nodo inicial (E) hasta el nodo objetivo (S). En este sentido, BFS y A* garantizan el camino más corto, mientras que DFS no lo asegura, generando soluciones subóptimas con entre 1.6× y 2.3× más pasos que el óptimo.

En cuanto a la eficiencia de exploración, medida como el número de nodos visitados, BFS explora casi la totalidad del grafo debido a su naturaleza de búsqueda ciega, sin utilizar información sobre la ubicación del destino. Por el contrario, A* logra reducir entre un 8% y un 22% el número de nodos explorados gracias al uso de una heurística basada en la distancia Manhattan. Esta ventaja es más evidente en laberintos complejos o de mayor tamaño, mientras que en estructuras simples o con poca ambigüedad la diferencia es menor. DFS, aunque explora menos nodos, lo hace a costa de la calidad de la solución.

Respecto al tiempo de ejecución, BFS suele ser el más rápido debido a su simplicidad, mientras que A* presenta un ligero costo adicional asociado a la gestión de una estructura de datos tipo heap para priorizar nodos. DFS también puede ser rápido en términos de tiempo, pero su falta de optimalidad limita su utilidad en este contexto.

En conclusión, la métrica principal para evaluar estos algoritmos es la optimalidad del camino, complementada por el tiempo de ejecución en escenarios donde la rapidez es crítica. La eficiencia en la exploración resulta útil para analizar el comportamiento interno del algoritmo, pero no impacta directamente en el resultado final para el usuario. Bajo estos criterios, A* se posiciona como el algoritmo más adecuado para resolver laberintos, especialmente a medida que aumenta la complejidad del grafo.

II. OPTIMIZACIÓN DE COLONIAS DE HORMIGAS

A. Correr la implementación planteada

El código original implementa el algoritmo ACO con varios problemas que no son evidentes a primera vista pero que impactan significativamente su funcionamiento, especialmente en escenarios complejos como el Caso 2.

El primer punto a notar está en los parámetros iniciales. El original define alpha=0.1 y beta=15, lo que genera un desbalance severo en la fórmula de probabilidad. Como beta es 150 veces más grande que alpha, la heurística de distancia euclidiana al destino domina completamente la decisión de cada hormiga, mientras que las feromonas acumuladas son prácticamente irrelevantes. Esto significa que el algoritmo se comporta como un simple greedy que siempre avanza hacia el vecino más cercano al destino, sin aprovechar el aprendizaje colectivo que es la esencia de ACO. La versión corregida usa alpha=1.0 y beta=5.0, logrando un balance real donde tanto las feromonas como la heurística influyen en la decisión.

El segundo problema está en _deposit_pheromones(). El original deposita siempre +1 por celda sin importar la calidad del camino, lo que significa que un camino de 8 pasos y uno de 20 pasos reciben exactamente el mismo refuerzo por celda. Esto elimina cualquier señal de calidad en las feromonas. La versión corregida usa 100 / len(path), de modo que caminos más cortos depositan más feromona y el algoritmo aprende a preferirlos con el tiempo.

El tercer y más crítico problema está en find_best_path(). El original ordena todos los caminos por longitud y toma el más corto, sin verificar si ese camino realmente llegó al destino. Cuando una hormiga queda atrapada y hace break, su camino incompleto de 3 o 4 pasos entra al ranking y gana automáticamente por ser el más corto. En el Caso 1 esto no se nota porque la barrera es parcial y la mayoría de hormigas llegan al destino. Pero en el Caso 2, donde la barrera bloquea casi todo el paso, la mayoría de hormigas quedan atrapadas y sus caminos incompletos envenenan las feromonas en celdas incorrectas, haciendo que el algoritmo nunca converja. La corrección es filtrar primero con valid_paths = [p for p in all_paths if p[-1] == self.end] antes de ordenar por longitud.

Finalmente, el original solo deposita feromonas en el mejor camino de cada iteración, desperdiciando la información de todas las demás hormigas que también llegaron al destino. La versión corregida deposita en todos los caminos válidos, enriqueciendo el aprendizaje colectivo en cada iteración.

B. ¿Qué ocurre con el segundo caso de estudio?

1. Al escoger el mejor camino una condición está faltando, ¿es suficiente elegir el camino con el menor tamaño?

El problema se encontraba en la función `find_best_path()`. En la implementación original, el algoritmo ordenaba todos los caminos generados únicamente en función de su longitud y seleccionaba el más corto, sin verificar si dicho camino alcanzaba efectivamente el nodo objetivo.

En el Caso de Estudio 2, existe una barrera que bloquea las columnas 0, 1, 2 y 3 en la fila y=2. Como consecuencia, muchas hormigas quedan atrapadas y terminan generando caminos incompletos de apenas 3 o 4 pasos. Dado que el criterio de selección se basaba únicamente en la longitud, estos caminos incompletos eran considerados como los “mejores” por ser los más cortos.

Esto generaba un efecto negativo en el algoritmo, ya que las feromonas se reforzaban en rutas incorrectas. Como resultado, el sistema convergía hacia soluciones inválidas, por ejemplo:
[(0,0),(1,0),(1,1),(0,1)] el cual es un camino que nunca alcanza el nodo objetivo (4,7).

La corrección consistió en modificar el criterio de selección del mejor camino. En lugar de considerar todos los caminos, se filtraron únicamente aquellos que llegan efectivamente al nodo destino. Posteriormente, entre estos caminos válidos, se selecciona el de menor longitud.

Este cambio asegura que el algoritmo optimice únicamente sobre soluciones factibles, evitando reforzar caminos incompletos.

Este error corresponde a una mala definición de la función objetivo, al no restringir las soluciones a aquellas que alcanzan el nodo final.

2. **Cambiar el número de hormigas, cambiar los parámetros: taza de evaporación, Alpha, Beta.**

A continuación se presentan los cambios realizados en los parámetros del modelo y su justificación:

Parámetro	Valor Original	Valor Corregido	Justificación
`num_ants`	10	30	Un mayor número de hormigas incrementa la exploración del espacio de búsqueda, aumentando la probabilidad de encontrar caminos que rodeen la barrera.
`evaporation_rate`	0.1	0.3	Una mayor tasa de evaporación reduce la persistencia de feromonas en rutas incorrectas, evitando que el algoritmo se quede atrapado en soluciones subóptimas.
`alpha`	0.1	1.0	Con un valor muy bajo de alpha, la influencia de las feromonas era casi nula, impidiendo el aprendizaje colectivo. Al aumentarlo, las rutas exitosas tienen mayor impacto en decisiones futuras.
`beta`	15	5.0	Un valor excesivamente alto de beta hacía que el algoritmo se comportara de forma casi determinista (greedy), priorizando únicamente la distancia e ignorando las feromonas. Reducirlo permite un mejor equilibrio entre exploración y explotación.

Los ajustes realizados permiten corregir un problema de optimización mal definida y mejorar el equilibrio entre exploración y explotación, lo que resulta en una convergencia más estable hacia soluciones válidas.

C. Describir los parámetros del modelo

El modelo ACO recibe varios parámetros que en conjunto definen tanto el entorno de búsqueda como el comportamiento de las hormigas.
start y end simplemente definen el punto de origen y el destino en la grilla. Todas las hormigas nacen en start y el algoritmo termina exitoso cuando alguna llega a end.

obstacles es la lista de celdas bloqueadas que ninguna hormiga puede pisar. Simulan paredes o barreras físicas en el entorno, y el método _get_neighbors() se encarga de excluirlas automáticamente al calcular los movimientos válidos.
grid_size define el tamaño del espacio de búsqueda, en este caso una grilla de 10x10. A mayor grilla, mayor complejidad computacional porque hay más celdas posibles que explorar.

num_ants controla cuántas hormigas exploran en paralelo en cada iteración. Con pocas hormigas el algoritmo explora poco el espacio y puede no encontrar caminos válidos, especialmente cuando hay barreras complejas como en el Caso 2. Aumentarlo a 30 fue clave para que suficientes hormigas lograran rodear la barrera.

evaporation_rate determina qué tan rápido desaparecen las feromonas en cada iteración, aplicando pheromones *= (1 - evaporation_rate). Un valor bajo hace que las feromonas persistan mucho y el algoritmo converja rápido pero arriesgándose a quedarse atascado en una ruta subóptima. Un valor más alto como 0.3 hace que el algoritmo olvide más rápido las rutas malas y explore con más libertad.
alpha es el exponente que controla el peso de las feromonas en la fórmula de probabilidad. En el código original estaba en 0.1, lo que prácticamente anulaba la influencia de las feromonas y hacía que el aprendizaje colectivo entre iteraciones no funcionara. Al subirlo a 1.0 las feromonas pasan a tener un peso real en la decisión de cada hormiga.

beta es el exponente que controla el peso de la heurística, que en este caso es la distancia euclidiana al destino. Con el valor original de 15 y un alpha de apenas 0.1, la heurística dominaba completamente la decisión y el algoritmo se comportaba como un simple greedy que siempre elige el vecino más cercano al destino, ignorando por completo las feromonas acumuladas. Al bajarlo a 5.0 se logra un balance real entre seguir el rastro de feromonas y avanzar hacia el destino.

Finalmente, pheromones es la matriz que actúa como memoria colectiva del algoritmo. Se inicializa en 1.0 para todas las celdas y se va actualizando en cada iteración: las celdas que forman parte de buenos caminos acumulan más feromona, atrayendo a las hormigas siguientes hacia esas rutas. Es el mecanismo central que hace que ACO aprenda con el tiempo.

D. ¿Qué es Random Search y Grid Search? ¿Cómo aplicarlos para esta heurística?

El modelo ACO recibe varios parámetros que en conjunto definen tanto el entorno de búsqueda como el comportamiento de las hormigas.
start y end simplemente definen el punto de origen y el destino en la grilla. Todas las hormigas nacen en start y el algoritmo termina exitoso cuando alguna llega a end.

obstacles es la lista de celdas bloqueadas que ninguna hormiga puede pisar. Simulan paredes o barreras físicas en el entorno, y el método _get_neighbors() se encarga de excluirlas automáticamente al calcular los movimientos válidos.
grid_size define el tamaño del espacio de búsqueda, en este caso una grilla de 10x10. A mayor grilla, mayor complejidad computacional porque hay más celdas posibles que explorar.

num_ants controla cuántas hormigas exploran en paralelo en cada iteración. Con pocas hormigas el algoritmo explora poco el espacio y puede no encontrar caminos válidos, especialmente cuando hay barreras complejas como en el Caso 2. Aumentarlo a 30 fue clave para que suficientes hormigas lograran rodear la barrera.

evaporation_rate determina qué tan rápido desaparecen las feromonas en cada iteración, aplicando pheromones *= (1 - evaporation_rate). Un valor bajo hace que las feromonas persistan mucho y el algoritmo converja rápido pero arriesgándose a quedarse atascado en una ruta subóptima. Un valor más alto como 0.3 hace que el algoritmo olvide más rápido las rutas malas y explore con más libertad.
alpha es el exponente que controla el peso de las feromonas en la fórmula de probabilidad. En el código original estaba en 0.1, lo que prácticamente anulaba la influencia de las feromonas y hacía que el aprendizaje colectivo entre iteraciones no funcionara. Al subirlo a 1.0 las feromonas pasan a tener un peso real en la decisión de cada hormiga.

beta es el exponente que controla el peso de la heurística, que en este caso es la distancia euclidiana al destino. Con el valor original de 15 y un alpha de apenas 0.1, la heurística dominaba completamente la decisión y el algoritmo se comportaba como un simple greedy que siempre elige el vecino más cercano al destino, ignorando por completo las feromonas acumuladas. Al bajarlo a 5.0 se logra un balance real entre seguir el rastro de feromonas y avanzar hacia el destino.

Finalmente, pheromones es la matriz que actúa como memoria colectiva del algoritmo. Se inicializa en 1.0 para todas las celdas y se va actualizando en cada iteración: las celdas que forman parte de buenos caminos acumulan más feromona, atrayendo a las hormigas siguientes hacia esas rutas. Es el mecanismo central que hace que ACO aprenda con el tiempo.

E. Pregunta de investigación:

Paso 1 — Representar el problema como grafo. Cada ciudad es un nodo y cada par de ciudades tiene una arista con peso igual a la distancia entre ellas. La matriz de distancias reemplaza a la grilla de feromonas del ejercicio.

Paso 2 — Inicializar la matriz de feromonas. En lugar de una grilla 2D, se usa una matriz n x n donde pheromones[i][j] representa la feromona en la arista entre la ciudad i y la ciudad j. Se inicializa uniformemente igual que en el ejercicio.

Paso 3 — Construir soluciones. Cada hormiga parte de una ciudad aleatoria y en cada paso elige la siguiente ciudad no visitada usando la misma fórmula probabilística del ejercicio, donde la heurística ahora es 1 / distancia(i, j) en lugar de la distancia euclidiana al destino.

Paso 4 — Aplicar restricción de visita única. A diferencia del ejercicio donde visited solo evitaba ciclos, aquí visited es obligatorio y estricto: una hormiga nunca puede volver a una ciudad ya visitada. Cuando ha visitado todas las ciudades, cierra el tour regresando a la ciudad inicial.

Paso 5 — Evaluar y actualizar feromonas. Al final de cada iteración se calcula la distancia total de cada tour. Se evapora la feromona global y se deposita en las aristas de los mejores tours, usando Q / distancia_total como cantidad de depósito.

III. ENSAYO

Modelos de lenguaje y algoritmos de búsqueda: ¿oposición o convergencia conceptual?

En el campo de la inteligencia artificial, dos grandes enfoques han marcado la evolución en la resolución de problemas: los algoritmos de búsqueda, propios de la IA clásica, y los modelos de lenguaje basados en arquitecturas Transformer, que representan el paradigma dominante en la actualidad (Russell & Norvig, 2021; Vaswani et al., 2017). A primera vista, ambos parecen pertenecer a mundos completamente distintos: uno explora grafos de estados con reglas explícitas; el otro aprende distribuciones de probabilidad sobre millones de textos.
Sin embargo, esta separación superficial oculta una pregunta más profunda, y más interesante: ¿están haciendo algo fundamentalmente diferente, o se trata de dos caras de la misma moneda? Desde la perspectiva de la Ciencia de Datos, responder esta pregunta importa en la práctica: condiciona cómo interpretamos la inferencia de un modelo, cómo elegimos entre soluciones para un problema, y cómo podríamos diseñar sistemas más poderosos combinando lo mejor de cada enfoque. Este ensayo argumenta que, pese a sus diferencias técnicas, los algoritmos de búsqueda y los modelos de lenguaje son conceptualmente convergentes: comparten el esqueleto de explorar un espacio hacia un objetivo, aunque lo hagan de formas radicalmente distintas. La convergencia no es solo teórica; se materializa en técnicas concretas como el *beam search*, y abre la puerta a sistemas híbridos que ya están redefiniendo el estado del arte.

I. Algoritmos de búsqueda: exploración estructurada y optimización explícita

Los algoritmos de búsqueda representan problemas como grafos dirigidos: los nodos son estados posibles del mundo, las aristas son acciones o transiciones, y la meta es encontrar un camino óptimo desde un estado inicial hasta uno objetivo, según una función de costo bien definida (Russell & Norvig, 2021). Esta representación tiene una virtud fundamental: *todo es transparente*. Sabemos exactamente qué se está explorando, por qué se elige un camino y cuándo se ha llegado a la solución. El algoritmo BFS (*Breadth-First Search*) garantiza encontrar el camino más corto en grafos no ponderados, explorando todos los vecinos antes de avanzar un nivel. Sin embargo, su consumo de memoria crece exponencialmente con la profundidad del grafo.
El algoritmo A* (Hart, Nilsson & Raphael, 1968) resolvió en gran medida este problema introduciendo una función heurística h(n) que guía la búsqueda hacia regiones más prometedoras del espacio de estados, reduciendo dramáticamente el número de nodos que es necesario explorar. Siempre que la heurística sea *admisible* (es decir, nunca sobreestime el costo real), A* garantiza encontrar la solución óptima.
Desde la perspectiva de la Ciencia de Datos, los algoritmos de búsqueda clásicos son procesos de optimización discreta y determinista, con tres propiedades que los hacen especialmente valiosos en ciertos contextos: la función objetivo es explícita y auditable, la evaluación de cada estado es determinista, y el proceso completo es interpretable paso a paso. Esta interpretabilidad no es un lujo académico: en aplicaciones críticas como planificación médica, navegación autónoma o juegos de estrategia, poder explicar por qué el sistema tomó una decisión es tan importante como la decisión misma.
La limitación estructural de estos algoritmos es el crecimiento exponencial del espacio de búsqueda. En el ajedrez, por ejemplo, el número de posiciones posibles supera el número de átomos en el universo observable. Incluso con heurísticas excelentes, la búsqueda exhaustiva se vuelve inviable para dominios de alta dimensionalidad o con reglas imprecisas, como el lenguaje natural. Fue precisamente esta limitación la que motivó el surgimiento de un enfoque radicalmente distinto.

II. Modelos de lenguaje: aprendizaje probabilístico e inferencia en espacios latentes

Los modelos de lenguaje basados en la arquitectura Transformer (Vaswani et al., 2017) operan sobre un principio completamente diferente. En lugar de definir un espacio de estados explícito y navegarlo, aprenden una función paramétrica que captura la distribución de probabilidad del lenguaje directamente desde los datos:

$P(x_t \mid x_1, x_2, \ldots, x_{t-1})$

Esta función se aprende minimizando la entropía cruzada entre las predicciones del modelo y el texto real, usando variantes de *gradient descent* sobre conjuntos de datos de escala masiva (Goodfellow, Bengio & Courville, 2016). El resultado es un modelo con miles de millones de parámetros que ha internalizado, de forma implícita, enormes cantidades de conocimiento sobre gramática, hechos del mundo, razonamiento y estilo.

El mecanismo central que hace posible esto es la atención multi-cabeza (*multi-head self-attention*), que permite al modelo ponderar dinámicamente qué partes del contexto previo son relevantes para predecir el siguiente token. A diferencia de las redes recurrentes anteriores, Transformer procesa toda la secuencia en paralelo, lo que permite escalar el entrenamiento a corpus de billones de palabras (Brown et al., 2020). Modelos como GPT-3 y sus sucesores demostraron que esta escala genera capacidades emergentes que no estaban explícitamente programadas: razonamiento aritmético, traducción, programación y síntesis de argumentos.

Durante la inferencia, el modelo genera texto de forma autoregresiva: produce un token a la vez, condicionando cada predicción en todos los tokens anteriores. La estrategia de decodificación determina qué token se selecciona en cada paso: *greedy decoding* toma siempre el token más probable; *sampling* muestrea de la distribución completa para generar variedad; y *beam search* mantiene un conjunto de las k secuencias más prometedoras en paralelo (Devlin et al., 2019). Esta última estrategia es, como veremos, el puente conceptual más importante entre los dos enfoques.

Desde la Ciencia de Datos, los LLM implican trabajar en espacios de alta dimensión donde no existe una función objetivo auditable en tiempo de inferencia, ni garantías formales de optimalidad. La fortaleza está en otro lugar: una capacidad de generalización que ningún sistema de búsqueda clásico puede igualar, y la habilidad de manejar la ambigüedad y el contexto abierto que caracterizan al lenguaje humano.

III. Convergencia: hacia una interpretación como búsqueda probabilística

A pesar de sus diferencias, es posible —y útil— establecer una convergencia conceptual entre ambos paradigmas. El *beam search* es el caso más ilustrativo: cuando un modelo de lenguaje genera texto usando esta estrategia, está literalmente realizando una búsqueda en árbol sobre el espacio de secuencias posibles, manteniendo las k hipótesis más probables en cada paso y expandiéndolas en paralelo. La diferencia con A* no es de naturaleza, sino de representación: en lugar de una función de costo definida por reglas, se usa la log-probabilidad acumulada como función objetivo.

Esta analogía permite reformular los LLM en términos que los algoritmos de búsqueda reconocerían:

- Espacio de estados: el conjunto de todas las secuencias de tokens posibles.
- Función objetivo: la probabilidad conjunta aprendida durante el entrenamiento.
- Estrategia de exploración: greedy, sampling o beam search.
- Solución: la secuencia generada que maximiza (aproximadamente) esa función.

Sin embargo, esta búsqueda tiene limitaciones importantes que la diferencian del ideal clásico. No es exhaustiva: el espacio de secuencias crece exponencialmente con la longitud, por lo que ninguna estrategia de decodificación realista puede explorar másque una fracción ínfima de él. Tampoco garantiza optimalidad: incluso con beam search, la secuencia encontrada puede no ser la de mayor probabilidad global (Stahlberg & Byrne, 2019). En ese sentido, los LLM implementan una *búsqueda heurística implícita* en un espacio probabilístico aprendido.

Esta convergencia conceptual no es solo un ejercicio intelectual: tiene implicaciones prácticas directas para la Ciencia de Datos. Comprender que la generación de texto es una forma de búsqueda ayuda a diagnosticar fallos del modelo (por ejemplo, la tendencia al texto repetitivo con greedy decoding), a elegir la estrategia de decodificación apropiada para cada tarea, y a diseñar sistemas más robustos que combinen la estructura explícita de la búsqueda con la flexibilidad del aprendizaje. Desarrollos recientes como *AlphaCode* y los modelos de razonamiento con cadena de pensamiento (*chain-of-thought*) son evidencia de esta dirección: integran planificación explícita dentro de la generación probabilística (Wei et al., 2022).

Conclusión

Los algoritmos de búsqueda y los modelos de lenguaje representan dos filosofías distintas pero complementarias dentro de la inteligencia artificial. Los primeros destacan por su rigor, interpretabilidad y garantías formales: sabemos exactamente qué buscan y por qué llegan a una solución. Los segundos destacan por su capacidad de aprendizaje, generalización y manejo de la complejidad abierta del lenguaje natural.

Desde la Ciencia de Datos, los modelos de lenguaje pueden entenderse como sistemas de búsqueda probabilística implícita: en lugar de navegar un grafo predefinido, navegan un espacio dedistribuciones aprendidas desde los datos, usando estrategias de decodificación que son, en esencia, algoritmos de búsqueda adaptados. Esta lectura no disuelve las diferencias reales entre ambos paradigmas, pero sí revela la unidad profunda que las subyace.

El futuro inmediato de la IA apunta hacia sistemas híbridos que integren la precisión de la búsqueda estructurada con la flexibilidad del aprendizaje automático. Arquitecturas como AlphaGo —que combina redes neuronales profundas con Monte Carlo Tree Search— o los modelos de razonamiento con planificación explícita son precursores de esta convergencia (Silver et al., 2017). Para un científico de datos, entender ambos paradigmas y sus puentes no es solo un ejercicio académico: es una ventaja competitiva real a la hora de diseñar, evaluar y mejorar sistemas de inteligencia artificial.

Bibliografía

[1] Brown, T., Mann, B., Ryder, N., et al. (2020). *Language Models are Few-Shot Learners.* Advances in
Neural Information Processing Systems (NeurIPS), 33, 1877–1901.
https://arxiv.org/abs/2005.14165

[2] Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). *BERT: Pre-training of Deep Bidirectional*
Transformers for Language Understanding. Proceedings of NAACL-HLT 2019, 4171–4186.
https://arxiv.org/abs/1810.04805

[3] Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning.* MIT Press.
https://www.deeplearningbook.org

[4] Hart, P. E., Nilsson, N. J., & Raphael, B. (1968). *A formal basis for the heuristic determination of minimum cost paths.* IEEE Transactions on Systems Science and Cybernetics, 4(2), 100–107.
https://doi.org/10.1109/TSSC.1968.300136

[5] Russell, S., & Norvig, P. (2021). Artificial Intelligence: A Modern Approach (4th ed.). Pearson.
https://aima.cs.berkeley.edu

[6] Silver, D., Schrittwieser, J., Simonyan, K., et al. (2017). *Mastering the game of Go without human*
knowledge. Nature, 550, 354–359.
https://doi.org/10.1038/nature24270

[7] Stahlberg, F., & Byrne, B. (2019). On NMT Search Errors and Model Errors: Cat Got Your Tongue?
Proceedings of EMNLP-IJCNLP 2019, 3356–3362.
https://arxiv.org/abs/1908.10090

[8] Vaswani, A., Shazeer, N., Parmar, N., et al. (2017). *Attention Is All You Need.* Advances in Neural
Information Processing Systems (NeurIPS), 30.
https://arxiv.org/abs/1706.03762

[9] Wei, J., Wang, X., Schuurmans, D., et al. (2022). *Chain-of-Thought Prompting Elicits Reasoning in*
Large Language Models. Advances in Neural Information Processing Systems (NeurIPS), 35.
https://arxiv.org/abs/2201.11903